Mock Lab Test 2C — COR-IS1704 (practice paper)

25 marks · 1 hour 30 minutes

Written for revision in the style of the 2023–26 Lab Test 2 papers. Not an official paper.

Instructions

1. Work in the `your_email_id` folder. Rename the folder to your email ID, but **do not rename any `.py` file**.
2. Write each function in the file named in the question. Use the matching `qXX_test.py` to test it. The marker uses different test cases, so do not hard-code answers.
3. A file that does not run scores 0. If something is unfinished, comment it out or put `pass` under it before time is up.
4. Your functions must **return** the result. Do not `print` it.
5. Start the timer before you open Question 1. Look at `solutions/` only after you finish.

Question 1 [8 marks]

Part A (Difficulty *) — `q1a.py`

A food guide stores hawker stalls as a list of tuples `(name, rating, num_reviews)`:

- `name` (str), `rating` (float, 0.0–5.0), `num_reviews` (int).

Implement `get_good_stalls(stalls, min_rating, min_reviews)`.

- Return a **new list** with the names of the stalls whose rating is **at least** `min_rating` **and** whose number of reviews is **at least** `min_reviews`, in their original order.
- Return `[]` if no stall qualifies.
- Return `None` if `stalls` is empty.
- Your code must not modify `stalls`.

```
stalls = [('Ah Huat Duck Rice', 4.6, 320), ('Uncle Lim Char Kway Teow', 4.8, 95),
          ('Mei Mei Chicken Rice', 4.4, 1500), ('Bala Satay', 4.6, 100)]
get_good_stalls(stalls, 4.5, 100)   # ['Ah Huat Duck Rice', 'Bala Satay']
get_good_stalls(stalls, 4.9, 0)     # []
get_good_stalls([], 4.0, 10)        # None
```

Part B (Difficulty **) — `q1b.py`

Implement `get_initials(names)`. `names` (str) holds several names separated by `;`. Inside a name, words are separated by **one or more** spaces, and there may be spaces at the start or end.

- Return the initials of each name in **upper case**, with the names joined by `-`.
- Skip empty names (for example between `;;`).
- Return `''` if there are no names.

```
get_initials('tan ah kow;lim bee hoon; ong')   # 'TAK-LBH-O'
get_initials('  mary   ann  ')                 # 'MA'
get_initials('Siti Nurhaliza;a;;b')            # 'SN-A-B'
get_initials('')                               # ''
```

Question 2 [6 marks]

A transport card system logs trips in a text file. The **first line is a header**. Each following line has four fields separated by `|`:

```
CARD|DATE|MODE|FARE
C001|2026-11-02|BUS|$1.19
C002|2026-11-02|MRT|$1.48
C001|2026-11-02|MRT|$0.99
C001|2026-11-03|BUS|$1.19
C003|2026-11-03|BUS|$2.05
```

`DATE` is `YYYY-MM-DD`, `MODE` is `BUS` or `MRT`, and `FARE` always starts with `$`. The file may contain only the header.

Part A (Difficulty *) — `q2a.py`

Implement `read_trips(filename)`. Return a list of tuples `(card, (year, month, day), mode, fare)` in file order, where `year`, `month` and `day` are `int` and `fare` is a `float`. Return `[]` if there are no trips.

```
read_trips('trips-1.txt')
# [('C001', (2026, 11, 2), 'BUS', 1.19), ('C002', (2026, 11, 2), 'MRT', 1.48),
...]
```

Part B (Difficulty **) — `q2b.py`

Implement `get_spending(filename, card, mode=None)`.

- `mode` is optional. If it is not given, count trips of **every** mode. If it is given, count only trips of that mode.
- Return a tuple `(count, total)` where `total` is the total fare rounded to 2 decimal places.
- Return `(0, 0.0)` if the card has no matching trips.
- You may import and use `read_trips` from `q2a.py`.

```
get_spending('trips-1.txt', 'C001')          # (3, 3.37)
get_spending('trips-1.txt', 'C001', 'BUS')   # (2, 2.38)
get_spending('trips-1.txt', 'C002', 'BUS')   # (0, 0.0)
```

Question 3 [5 marks]

A stored-value card file records one transaction per line, in time order, as `ACTION,CARD,AMOUNT`:

```
TOPUP,C001,10.00
PAY,C001,1.19
PAY,C002,1.48
TOPUP,C002,5.00
PAY,C001,9.00
PAY,C001,8.81
PAY,C002,1.48
PAY,C003,0.50
```

Rules:

- Every card starts with a balance of `0.0` the first time it appears in the file.
- `TOPUP` adds `AMOUNT` to the balance.
- `PAY` subtracts `AMOUNT` **only if the balance is at least `AMOUNT`**. Otherwise the payment is **rejected** and the balance does not change.
- Round the balance to 2 decimal places after every accepted transaction.
- The file may be empty.

Part A (Difficulty **) — `q3a.py`

Implement `get_balances(filename)`. Return a dictionary `{card: final_balance}` with the cards in the order they first appear in the file. Return `{}` for an empty file.

```
get_balances('wallet-1.txt')   # {'C001': 0.0, 'C002': 3.52, 'C003': 0.0}
```

(Line 3 is rejected because C002 has 0.0. Line 5 is rejected because C001 has only 8.81.)

Part B (Difficulty **/***) — `q3b.py`

Implement `get_rejected(filename)`. Return a dictionary `{card: [(line_number, amount), ...]}` listing every rejected payment, where `line_number` counts from 1 and `amount` is a float.

- Only cards with at least one rejected payment appear.
- Cards appear in the order of their first rejected payment; each card's list is in file order.
- Return `{}` if nothing is rejected.

```
get_rejected('wallet-1.txt')
# {'C002': [(3, 1.48)], 'C001': [(5, 9.0)], 'C003': [(8, 0.5)]}
```

Question 4 [6 marks]

A school books seminar rooms. Each line of a bookings file is `ID,ROOM,START,END` with times in 24-hour `HH:MM`:

```
B01,SR2-1,09:00,10:30
B02,SR2-1,10:30,12:00
B03,SR2-2,09:30,11:00
B04,SR2-1,10:00,11:00
B05,SR2-2,13:00,14:00
B06,SR2-2,10:45,13:15
B07,SR3-4,08:00,18:00
```

A booking uses its room from `START` up to but **not including** `END`. So B01 (until 10:30) and B02 (from 10:30) do **not** overlap. All bookings are on the same day, `START` is always before `END`, and the file may be empty.

Part A (Difficulty ***) — `q4a.py`

Two bookings **clash** if they are in the **same room** and their times overlap.

Implement `get_clashes(filename)`. Return a list of tuples `(id_a, id_b)`, one for every clashing pair, where `id_a`'s line comes before `id_b`'s line in the file. Order the list by the position of `id_a`, then by the position of `id_b`. Return `[]` if there are no clashes.

```
get_clashes('bookings-1.txt')
# [('B01', 'B04'), ('B02', 'B04'), ('B03', 'B06'), ('B05', 'B06')]
```

Part B (Difficulty ***) — `q4b.py`

The school may move bookings to **any** room. The fewest rooms needed is the largest number of bookings in use at the same moment.

Implement `get_rooms_needed(filename)`. Return a tuple `(rooms_needed, time)`:

- `rooms_needed` (int): the largest number of bookings in use at the same moment.
- `time` (str, `HH:MM`): the **earliest** time at which that many bookings are in use.
- Return `(0, None)` for an empty file.

```
get_rooms_needed('bookings-1.txt')   # (5, '10:45')
get_rooms_needed('bookings-3.txt')   # (2, '14:30')
```

At 10:45 in `bookings-1.txt`, B02, B03, B04, B06 and B07 are all in use.

Hint: the count can only go up when a booking starts, so you only need to check the start times.

— END OF PAPER —